

외부 모델 리뷰 — 무엇이 어디서 틀렸 나

August 30, 2026

CONTENTS

외부 모델 리뷰 — 무엇이 어디서 틀렸나	
1. 유형 다섯	
유형 A — 문서가 없는 것을 집행자로 든다	
유형 B — 화면이 믿으면 안 되는 데이터를 믿는다	
유형 C — 검사가 통과하면서 아무것도 안 본다	
유형 D — 정보가 자기 규칙을 어긴다	
유형 E — 오탐이 나서 곧 꺼질 검사	
2. 어디를 고쳤나	
3. 이 리뷰가 말해주는 것	

외부 모델 리뷰 — 무엇이 어디서 틀렸나

2026-08-30 · 커밋 `a726717`

게이트 11개가 전부 초록불인 상태에서 codex 에 세 축으로 리뷰를 넘겼다. **46건이 나왔고 오답이 0 건이었다.** 인용된 줄 번호를 전부 열어 확인했다.

이 문서는 그 46건을 **결함 유형별로** 묶는다. 목록이 아니라 유형이 값어치다 — 같은 유형이 다음에도 나오기 때문이다.

회차	축	건수	P1
1	검사 코드	17	6
2	정본↔문서 정합	14	9
3	화면·빌더	15	9

2회차가 가장 많은 P1 을 냈다. 문서 간 정합은 이 저장소의 검사가 가장 약한 축이고, 그 사실을 스스로 알고 있었으면서(현장 R2 가 6건으로 2위) 검사를 그쪽에 안 세웠다.

1. 유형 다섯

유형 A — 문서가 없는 것을 집행자로 든다

`PRD.md` 레드라인 표가 집행자를 이렇게 적고 있었다.

레드라인	PRD 가 적은 집행	실제
<code>n < 20</code> 셀은 값을 안 보인다	<code>test_suppression.py</code>	삭제됨 (#49)
모든 숫자가 단일 출처에서	<code>test_single_source.py</code>	삭제됨 (#49)

보호가 사라진 것은 아니다 — `test_build_reads_canon.py` 로 옮겼다. 사라진 것은 문서와 파일의 연결이고, 원인은 단순하다: 파일을 지우는 것과 그 파일을 가리키는 문서를 고치는 것이 다른 작업이라 앞만 했다.

이 유형이 위험한 이유는 대상이 규제 문서라는 것이다. 규제 검토자가 “억제가 검사로 보호된다” 고 읽고 그 검사가 없는 상태로 사할 있었다.

같은 유형: PRD 가 게이트를 넷이라 하고(정보는 여섯), 게이트 11개를 “ten” 이라 하고, 지운 `index.html` 을 배포 화면으로 서술하고, 기술문서 게이트 대응표가 다섯 행(산문은 여섯)이고, 2026-08-25 정정 노트가 `총계는 22개입니다` 로 끝나는데 두 줄 아래가 `23개` .

막은 방법: `check_no_dangling_enforcers` — 백틱으로 감싼 `이름.py` · `이름.sh` 와 ``make` `타깃`` 의 실재를 요구한다. 두 가지를 먼저 틀리고 배웠고 그것이 아래 유형 C 와 이어진다.

그리고 이 문서를 쓰는 중에 47번째가 나왔다. `AGENTS.md` 가 `make shots` 를 지원 명령으로 들고 있었고 그 타깃은 #49 에서 지웠다. 파일명만 보던 첫 판의 검사가 못 봤다 — **없는 명령을 지원한다**고 적는 것은 없는 검사를 집행자로 드는 것과 같은 결함이고, 유형을 정리하는 중에 그 유형의 새 사례가 나온 것이 이 문서의 값어치를 스스로 말한다.

유형 B — 화면이 믿으면 안 되는 데이터를 믿는다

입력	전	후
<code>expires_at: "not-a-date"</code>	ACTIVE	<code>MALFORMED</code> 차단
<code>access: "DENIED"</code>	truthiness 로 통과	allow-list
<code>score: null</code>	마지막 밴드가 받아 Priority	표시 안 함
정보 제목에 <code></title></code>	그대로 치환	빌드 거부

셋의 공통 원인은 **경계 조건을 “없음” 과 “잘못됨” 으로 나누지 않은 것이다**. `expires_at` 이 없는 것은 통과해야 하고(그 필드 이전 동의가 전부 막힌다) 읽을 수 없는 것은 막아야 하는데, `new Date("not-a-date") <= now` 가 `false` 라 둘이 같은 경로로 갔다.

`score: null → Priority` 는 정보가 금지한 문장 그대로다 — **그럴듯한 값으로 대체하지 않는다**. 마지막 밴드에 `min` 을 안 둔 것은 의도였고(그 아래 전부를 받아야 한다) 그 의도가 비슷자까지 받는 catch-all 이 됐다.

치환은 이스케이프가 아니라 **거부**로 갔다. 사람이 정보에 적는 값이므로 `<` 로 조용히 바뀌어 화면에 나가는 것보다 빌드가 죽는 것이 낫다.

유형 C — 검사가 통과하면서 아무것도 안 본다

이 저장소가 반복한 유형이고 현장 R1(9건, 1위)이다. 이번에 여섯 개가 더 나왔다.

검사	어떻게 공허했나
데이터 없음	문구와 이름만 봐서, 그 문구를 유지한 채 밴드를 넣으면 통과
점수 비노출	들을 심고 하나만 찾았다
동의 상태 문구	문서 전체를 봐서 다른 사람의 상태가 오답을 가렸다
<code>MEMBER_CASES</code> 완전성	최상위 키만 세어 <code>booking</code> 의 절반이 미검증
금지 용어 목록	flow-style 정규식 + 빈 목록 가드 없음 → 재서식하면 0 회
<code>check_design_records</code> 의 <code>EXPIRED</code>	기술문서의 <code>care_action.close_reason</code> 으로 만족

마지막 것이 이 문서를 쓰는 이유에 가깝다. 그 검사를 만들면서 커밋 메시지에 “두 문서를 대조하는 검사” 라고 적었고, 대조하는 대상이 동의와 무관한 동명 토큰이었다. 검사를 만든 같은 날에 그 검사가 공허했다.

그리고 개수 가드 자체가 이 유형이었다.

- 1판 손으로 더한 상수 → 단언을 늘릴 때마다 다시 계산, 두 번 틀림
- 2판 소스의 `checked += 1` 을 셈 → 보호 대상과 같은 문장을 세므로
단언+카운터를 함께 지우면 기대값도 줄어든다
- 3판 이름 집합 → 이름을 빼는 것은 `diff` 에 보이는 선언

2판을 쓸 때 “손으로 세지 않는다” 는 개선이라고 생각했고, 세는 대상이 보호 대상과 같다는 것을 안 봤다.

인용 면제가 검사를 끄는 문제도 이 유형이다. 문서 어딘가에 인용된 정확 문자열이 하나 있으면 접힌 문자열 검사가 문서 전체에서 꺼졌다. 그리고 유형 A 를 막으려고 만든

`check_no_dangling_enforcers` 의 첫 판이 같은 실수를 반복했다 — 삭제 노트가 있는 줄을 면제했고, PRD 행을 되돌리고 노트만 남기면 통과했다. 하루 안에 같은 결함을 두 번 만들었다.

해법은 같다: **살아 있는 주장과 그 주장의 기록을 구분한다**. 정정 표지 앞은 지금 하는 주장이고 뒤는 기록이므로, 앞만 검사한다.

유형 D — 정본이 자기 규칙을 어긴다

- `kpis[].source: HRIS · 834 eligibility` — 같은 파일의 `terminology.forbidden` 이 금지
- `artifacts.dashboard.path: index.html` — 지운 파일. #49 가 `surfaces` 만 고쳤다
- 기술문서 `canonical.consent` 스키마에 CMIA 필드 **0개** — 정본은 셋을 선언

원인은 **정본을 검사 대상으로 안 봤다는 것이다**. 정본은 검사의 기준이므로 대상이 아니라고 암묵적으로 다뤘고, 그래서 문서를 스캔하는 검사가 정본을 열지 않았다.

세 번째가 특히 위험했다. 백엔드 설계자가 법이 요구하는 것을 저장할 수 없는 스키마로 구현하는데 화면은 이미 만료를 평가하는 상태였다. **이건 새로 만든 검사가 잡았다** — 사람이 읽어서 찾은 것이 아니다.

유형 E — 오탐이 나서 곧 꺼질 검사

- 44px 단언이 `.tab{` 철자만 맞아 의미가 같은 `.tab {` 이 실패
- 834 표 스캔이 절 경계를 안 봐서 다른 절의 표가 오탐 후보

그리고 반대 방향으로, **범위가 좁아 놓치는 것**:

- 치환 값 범위 검사가 `employer` 템플릿만 읽어 `member` 템플릿의 하드코딩을 못 봤다
- 옛 값 검사가 “전보다 적다” 만 봐서 **부분 전파**를 놓쳤다 (한쪽 화면만 따라 움직여도 통과)

후자를 고칠 때 첫 시도가 상한을 정본 잔존 수로 잡았고 **기준선이 깨졌다** — 정본 값 하나가 템플릿 자리 여럿을 정당하게 채운다. 상한은 **여전히 그 값을 받아야 하는 템플릿 자리**의 수다.

2. 어디를 고쳤나

그룹	대상	PR
A+B	PRD 6곳 · 기술문서 4곳 · 현장 1곳 · 화면 안전 경로 4	#72
문서화 중 발견	<code>AGENTS.md</code> 의 <code>make shots</code> · 검사를 <code>make</code> 타깃까지 확장	이 PR
C+D+E	검사 9곳 · 정보 3곳 · 기술문서 스키마	#73

추가한 검사 셋

검사	무엇을 막는가
<code>check_no_dangling_enforcers</code>	문서가 없는 파일을 집행자로 드는 것 (유형 A)
<code>check_canon_obeyes_itself</code>	정보가 금지 용어를 쓰거나 없는 경로를 가리키는 것 (유형 D)
<code>check_design_records</code> (재작성)	구현 없는 설계가 두 문서에서 갈라지는 것

재현한 고장 **14건**. 각 고장을 심어 검사가 실패하는 것을 보고 커밋했다.

3. 이 리뷰가 말해주는 것

게이트 11개가 초록불인 것이 **결함 46건과 양립한다**. 그것이 이 문서의 결론이다.

검사는 자기가 보도록 쓰인 것만 본다. 이 저장소의 검사는 **한 문서 안의 값과 화면에 도달하는 값**을 보도록 쓰여 있었고, 그 둘은 잘 지켜졌다 — 2회차에서 나온 결함이 대부분 문서 **사이에** 있었던 것이 그 증거다.

그리고 **검사를 만드는 순간이 가장 위험하다**. 이번에 새로 만든 검사 둘이 만든 날에 공허했고 (`check_design_records` 의 `EXPIRED`, `check_no_dangling_enforcers` 의 면제 범위), 둘 다 “이 검사는 X 를 본다” 고 적은 문장이 참이 아니었다. 현장 G1 이 “산문 주장도 코드와 대조한다” 고 정한 이유가 그것인데, **G1 을 지킨다고 생각하면서 어겼다**.

외부 모델을 부른 것이 맞았다. 자체 검토로는 0건이 나왔을 것이다 — 46건 중 어느 것도 게이트가 잡지 못했고, 그 게이트를 내가 썼다.